Lab 3 memo: sparsity and reduced precision on a laptop CPU

Run IDs are cited by the unique 6-hex suffix of `run_id` in `results/runs.jsonl`.

One Apple M4 Pro, `qnnpack`. Latency: median, batch 256 (MLP/CNN) or full graph (GCN). Δ val: vs dense fine-tuned control (seeds 1–3, mean ± std) or FP32 parent (precision, one run).

Arch	Best trade-off	Δ val	Bytes	Latency	Runs
MLP	S2 structured, 53%, FP32	−0.45 pp (0.8985±0.0012 vs 0.9029±0.0013)	440,661 vs 943,589	0.069 vs 0.149 ms	(43c7f6, cd6e77, 9e86f2, 08f3e5, 5e6d5d, ee44e5, f1db26)
CNN	Static INT8 (FX)	−0.28 pp (0.9154 vs 0.9182)	127,291 vs 472,607	3.35 vs 6.73 ms	(f082d1, 185cdf)
GCN	S3 90%, stored as CSR, densified to run	−0.13 pp (0.7947±0.0012 vs 0.7960±0.0020)	21,793 vs 94,741	1.25 vs 1.25 ms	(ad1dc9, b5e43b, fb277b, 8531cf, 4c8b06, 793d3d, 15a44b, 46d95b)

1. Best trade-offs. Only MLP S2 cuts both bytes and latency; MLP static INT8 is smaller and lossless but 3.1× slower (0.404 vs 0.130 ms; 25d6f7, 844c70). CNN static INT8 beats CNN S2 (5.26 vs 6.81 ms, −0.43 pp; ae7582, d7e1bd, 12e9c1, 185cdf, d18606). No GCN option was both accurate and faster: S2 50% is 1.25× faster but −1.5 pp (e33eed, b9f801, 513069, d8f495).

2. Fewer nonzeros, not faster. As predicted, CSR matmul was 3.2× (d5fa24) to 91× (704ff3) slower than dense: tuned dense GEMM beats index-chasing on layers this small, and the zeros-in-place path still multiplies zeros. With int32 indices CSR breaks even on size at 50% (944,648 vs 943,589 B; f2b98a), not my predicted 60–67%. S4 has no CPU semi-structured kernel and falls back to dense (6.778 vs 6.775 ms; 0daff6). Fewer bytes failed too: CNN FP16 halves size but runs 18.2 vs 6.73 ms (34979c).

3. Architecture. I predicted GCN > CNN > MLP; the answer depends on structure. Unstructured, the GCN was *least* sensitive (−0.13 pp at 90%, Table; predicted −3 to −5). Structured/N:M, it collapsed: S2 75% at 0.434 (94be4d), 1:4 at 0.484 (064215), because both remove width from a 16-wide bottleneck that 140 labels cannot retrain. The CNN fell before fine-tuning (1:4 at 0.276; 9fce5d) but abundant data recovered it to 0.893 (b58941). INT4 cost CNN 1.76, GCN 1.4, MLP 0.72 pp (002cb7, 2f7015, 143754); MLP least, as predicted.

4. Composition. Accuracy composed (sparse+INT8 0.8936 vs sparse FP32 0.8924; 5ad684, ae0892); size and speed did not. INT8 is stored dense, so sparse+INT8 equals dense INT8 in size (243,001 vs 242,981 B; e2d198) and speed (GCN 1.21 vs 1.24 ms, FP32 1.07; 24dba8, 5b95aa, bcec0f). That is backend-bound: `qnnpack` is the only engine, with no sparse INT8 kernel, PT2E or torchao. Coverage matters too: CNN dynamic INT8 quantizes 6.1% of MACs and is slower than FP32 (7.29 ms; 9e1d6e); static quantizes 100% and is the only meaningfully faster reduced format.

5. Laptop. MLP: S2 FP32. CNN: static INT8 for batched inference. GCN: dense FP32; at 95 KB (bcec0f) compression buys little. Uncertainties: - One ARM machine, `qnnpack` only; x86 fbgemm/oneDNN may differ. - At batch 1, CNN static INT8 is slower (0.182 vs 0.093 ms; f082d1, 185cdf). - Part C rows are single runs. - Confirmation seeds share one pruned parent; std is fine-tuning noise only. - INT4 is simulated; dequantization is cached before timing, so latency is FP32 compute. - GCN validation has 500 nodes (0.2 pp each); most GCN gaps are 1–3 nodes.

Reflection. Pi ran all 51 pilot arms to `ok` with no reruns and logged append-only, so every superseded result stays traceable. I overruled Pi's emulated INT8 (hand-written Python int64 matmuls) and required native `qnnpack` modules, which is what produced the CNN speedup. The frozen rule picked dynamic INT8 for the CNN by 14 of 5,000 validation images, and trading 0.28 pp for a 2× speedup was a deployment judgment that no extra code could settle.